

RStudio

Working with R – RStudio

RStudio is an Integrated Development Environment (IDE) for R and it helps you:

- write code - makes suggestions
- view the output of your code, including plots
- find errors
- manage files
- view documentation

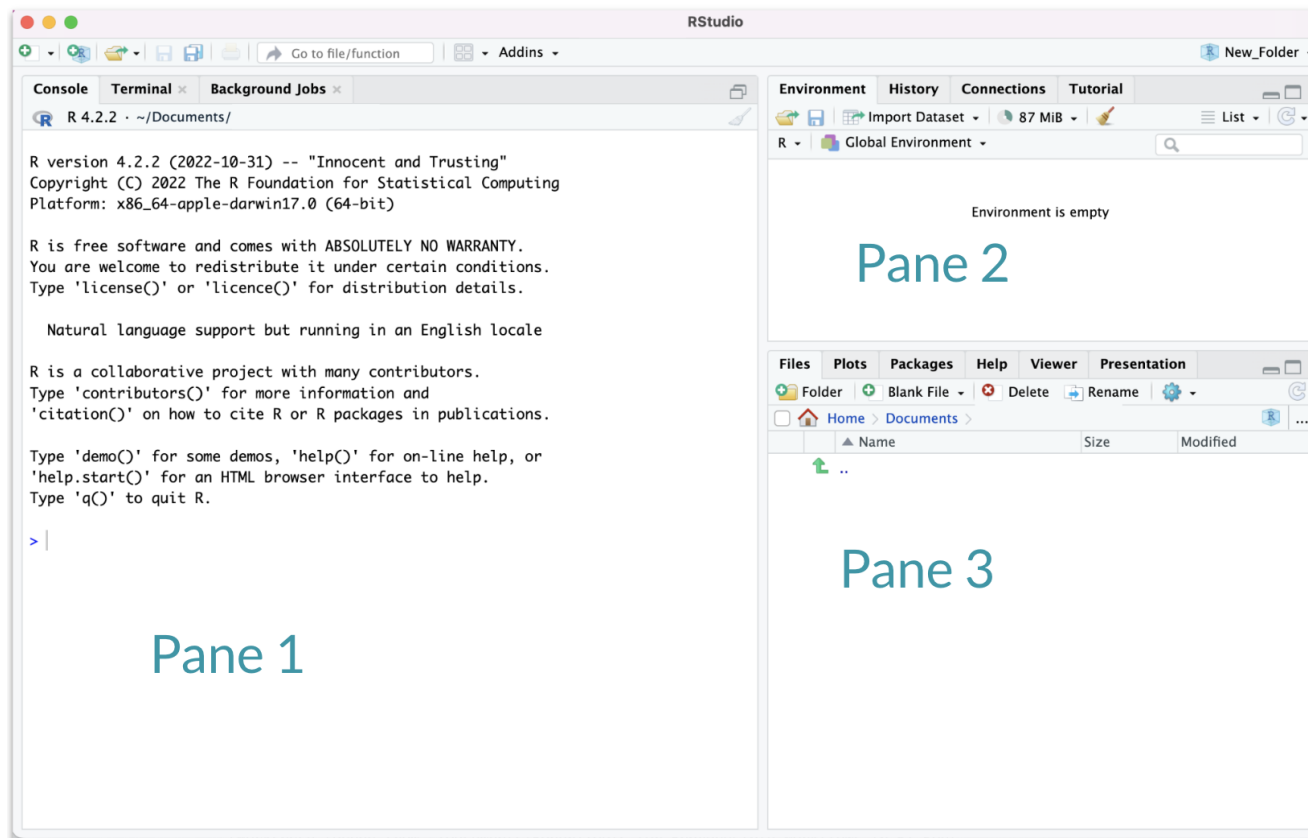


[\[source\]](#)

RStudio used to be the name of a company that is now called [Posit](#).

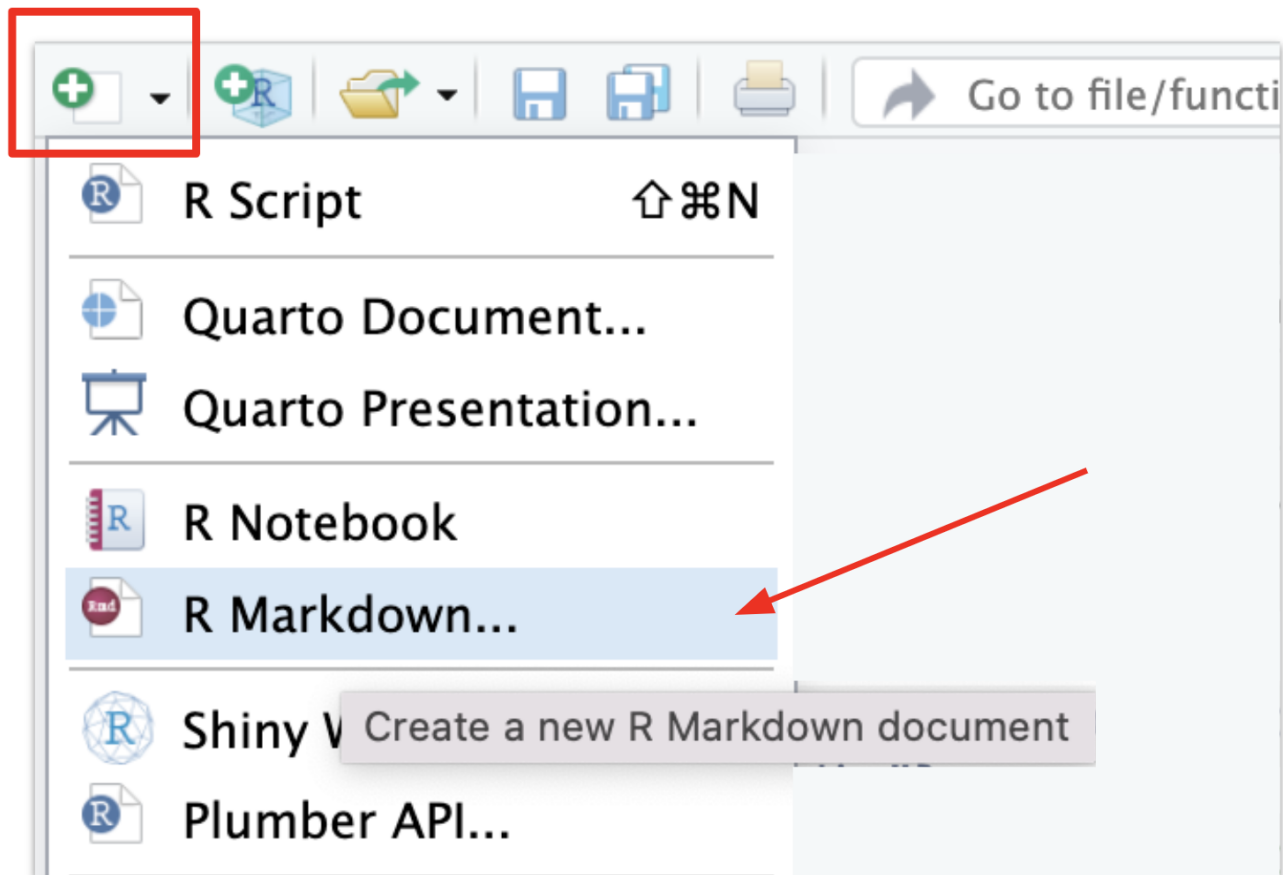
RStudio

First it is important to be familiar with the layout. When you first open RStudio, you will see 3 panes.



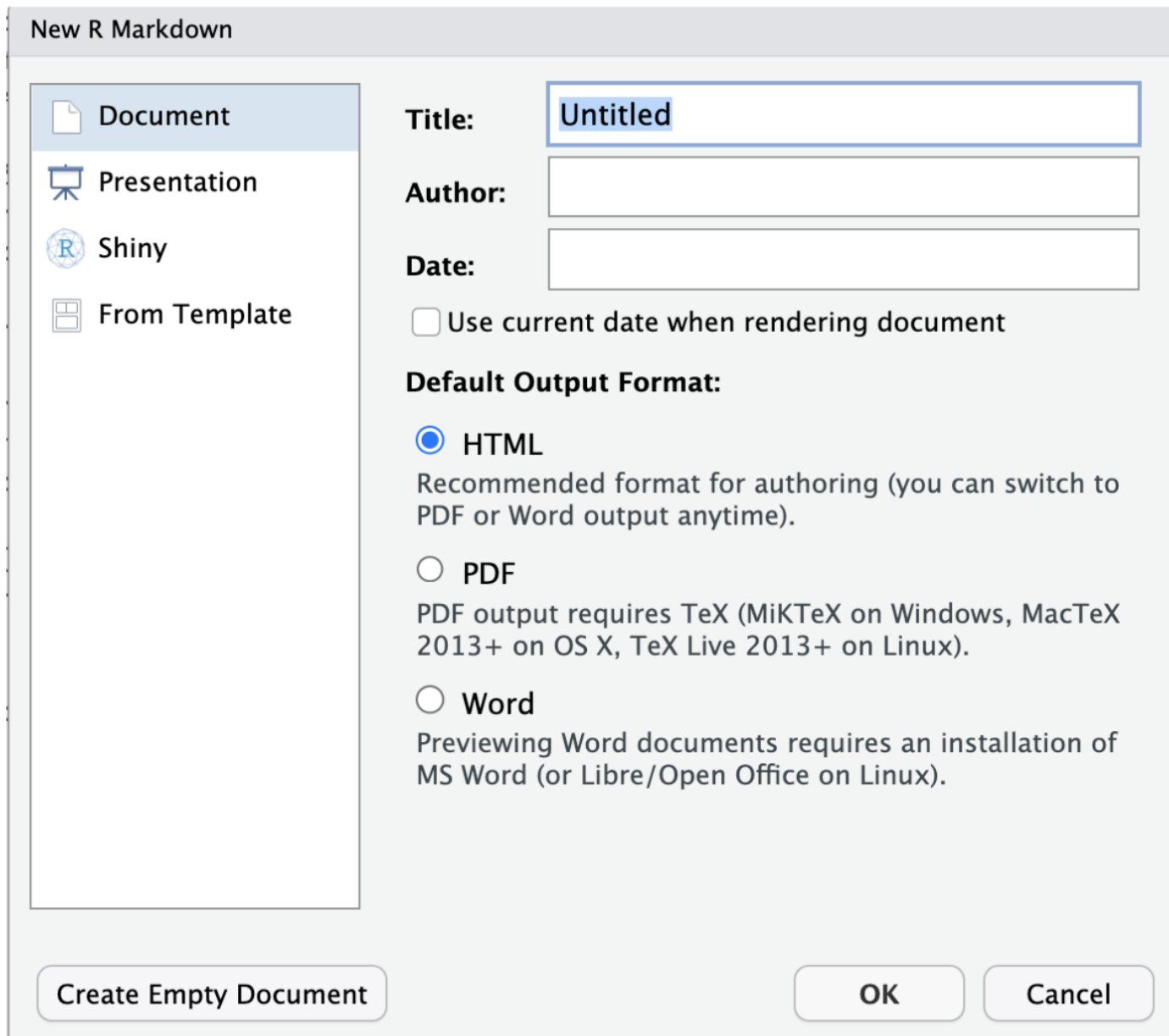
Hidden Pane

To save a copy of your code. You must open a file first - this will open a 4th pane. These files include Scripts or what are called R Markdown files.



Hidden Pane

You will see a popup that you can just say “OK” to for now.



The image shows a 'New R Markdown' dialog box. On the left is a sidebar with four options: 'Document' (selected), 'Presentation', 'Shiny', and 'From Template'. The 'Document' option is highlighted with a blue background. To the right of the sidebar are input fields for 'Title' (containing 'Untitled'), 'Author', and 'Date'. Below these is a checkbox labeled 'Use current date when rendering document'. Further down is the 'Default Output Format' section, which has three radio button options: 'HTML' (selected), 'PDF', and 'Word'. Each option has a descriptive text block below it. At the bottom of the dialog are three buttons: 'Create Empty Document', 'OK', and 'Cancel'.

New R Markdown

☒ Document

☐ Presentation

☐ Shiny

☐ From Template

Title:

Author:

Date:

☐ Use current date when rendering document

Default Output Format:

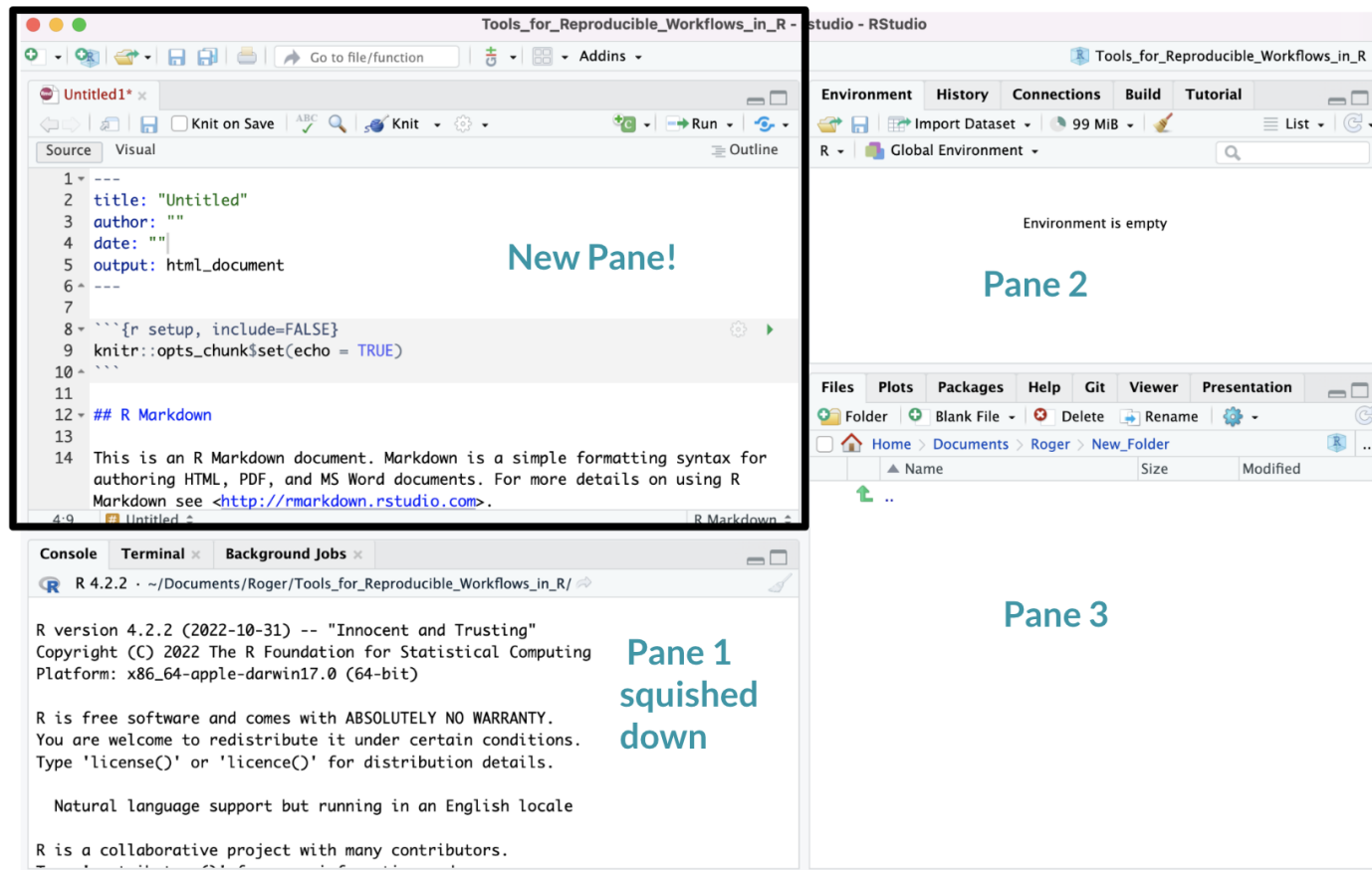
☒ **HTML**
Recommended format for authoring (you can switch to PDF or Word output anytime).

☐ **PDF**
PDF output requires TeX (MiKTeX on Windows, MacTeX 2013+ on OS X, TeX Live 2013+ on Linux).

☐ **Word**
Previewing Word documents requires an installation of MS Word (or Libre/Open Office on Linux).

Hidden Pane

Nice! now we have a place to save code! This is where we will mostly be working.



Working with R in R Studio - 2 major panes:

1. The **Source/Editor**:

- Top by default
- **saves your code**

2. The **R Console**:

- Bottom by default
- Calculator
- Installing packages `install.packages("package_name")`
- Place to try things out, then add to your editor
- **doesn't save your code**

RStudio

Super useful “cheatsheet”: [LINK](#)

Write Code

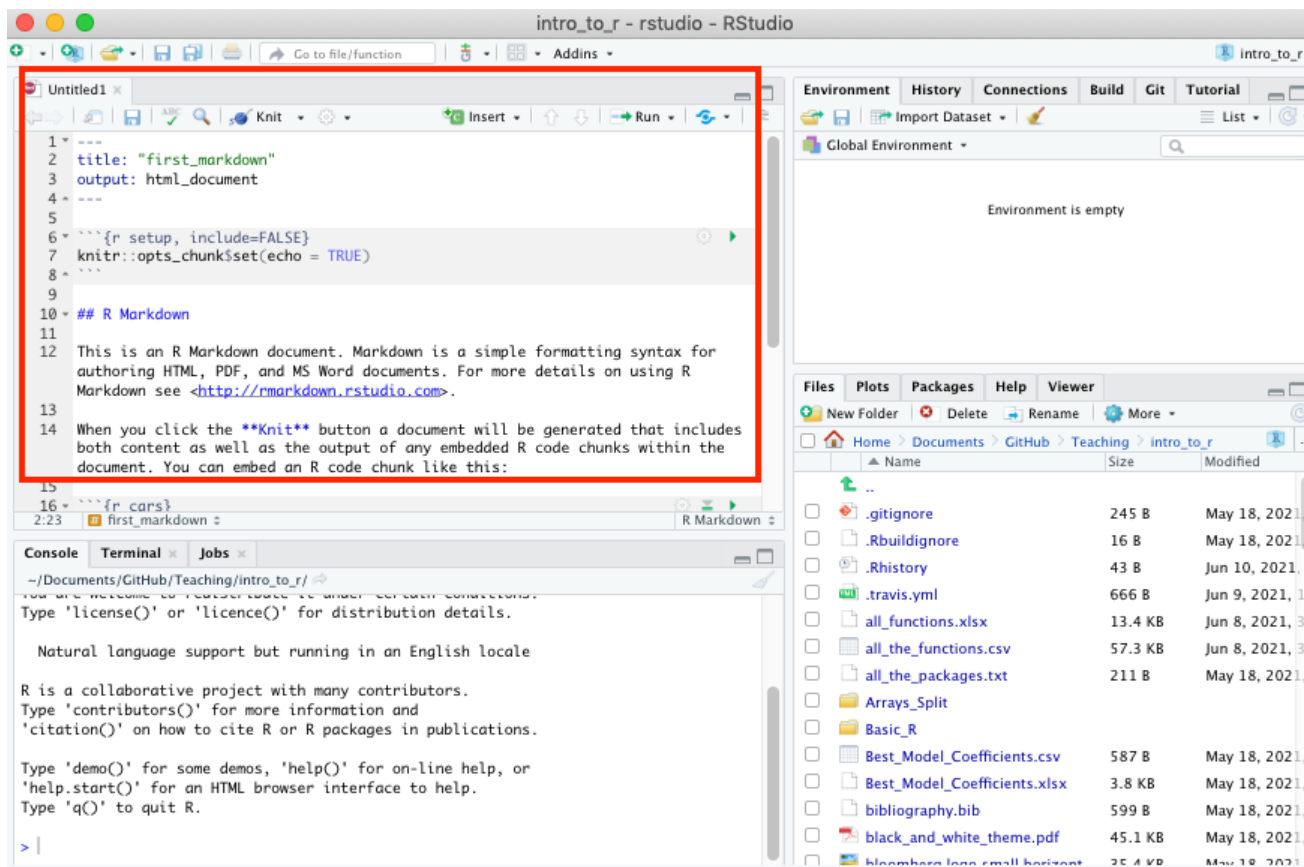
Navigate tabs
Open in new window
Save
Find and replace
Compile as notebook
Run selected code

The screenshot shows the RStudio interface with several panels and annotations. The main editor window displays R code with syntax highlighting and line numbers. The Environment panel on the right shows the current workspace with objects like 'iris' and 'foo'. The Files panel at the bottom shows the file browser. Annotations point to various features and actions:

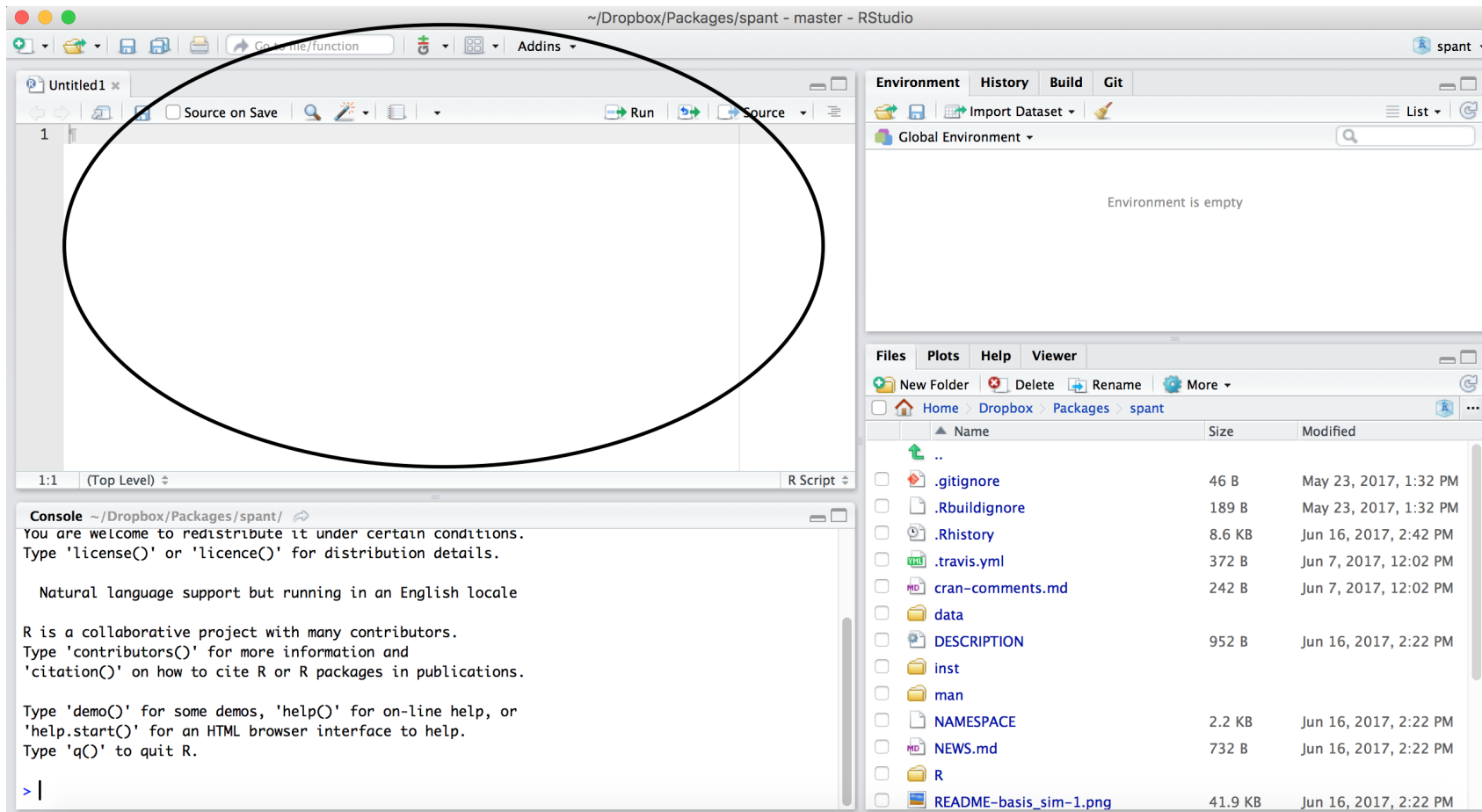
- Write Code:**
 - Navigate tabs
 - Open in new window
 - Save
 - Find and replace
 - Compile as notebook
 - Run selected code
- R Support:**
 - Import data with wizard
 - History of past commands to run/copy
 - Display .Rpres slideshows
 - File > New File > R Presentation
- Code Editor:**
 - Cursors of shared users
 - Re-run previous code
 - Source with or without Echo
 - Show file outline
 - Multiple cursors/column selection with Alt + mouse drag
 - Code diagnostics that appear in the margin. Hover over diagnostic symbols for details.
 - Syntax highlighting based on your file's extension
 - Tab completion to finish function names, file paths, arguments, and more.
 - Multi-language code snippets to quickly use common blocks of code.
 - Jump to function in file
 - Change file type
- Environment Panel:**
 - Load workspace
 - Save workspace
 - Delete all saved objects
 - Search inside environment
 - Choose environment to display from list of parent environments
 - Display objects as list or grid
- Data and Functions:**
 - Displays saved objects by type with short description
 - View in data viewer
 - View function source code
- Files Panel:**
 - Create folder
 - Upload file
 - Delete file
 - Rename file
 - Change directory
 - Path to displayed directory
 - A File browser keyed to your working directory. Click on file or directory name to open.
- Console:**
 - Working Directory
 - Maximize, minimize panes
 - Press ↑ to see command history
 - Drag pane boundaries

R Markdown files look different from scripts

It will look like this with text in it.



Scripts will just be empty

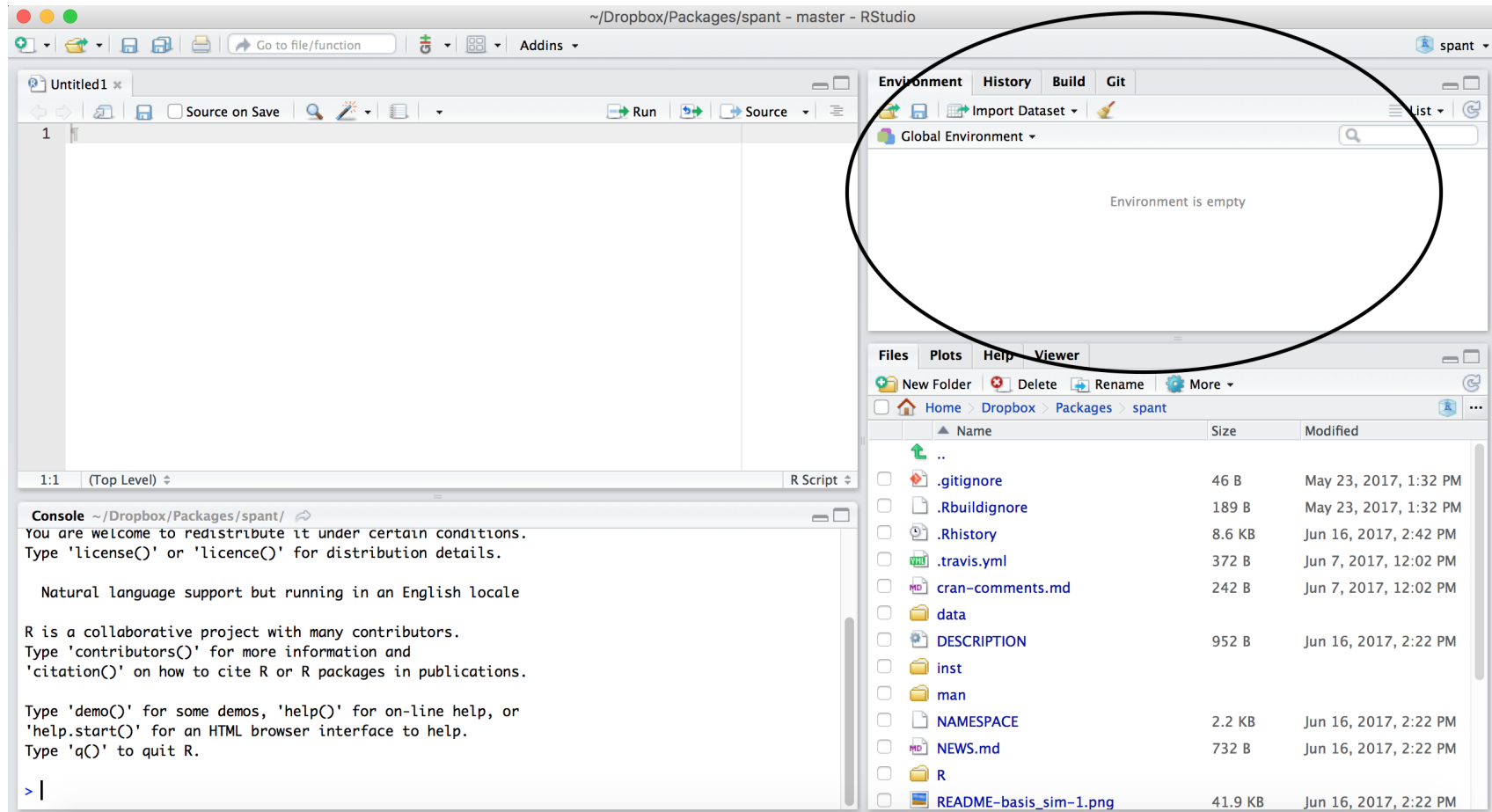


Scripts and R Markdown

Although people will use scripts often, and they are good for more programmatic purposes, we generally don't recommend them for Public Health Researchers.

For data analyses, R Markdown files are generally superior because they allow you to check your code and write more info about your code.

Workspace/Environment



Workspace/Environment

- Tells you what **objects** are in R
- What exists in memory/what is loaded?/what did I read in?

History

- Shows previous commands. Good to look at for debugging, but **don't rely** on it.
Instead use RMarkdown!
- Also type the “up” key in the Console to scroll through previous commands

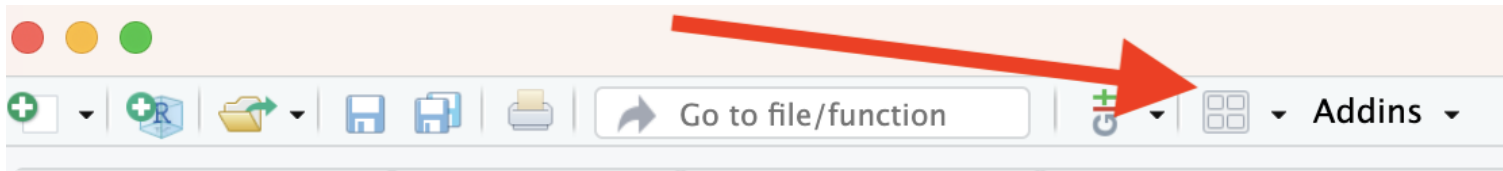
Lower Right Pane

- **Files** - shows the files on your computer of the directory you are working in
- **Viewer** - can view data or R objects
- **Help** - shows help of R commands
- **Plots** - pictures and figures
- **Packages** - list of R packages that are loaded in memory

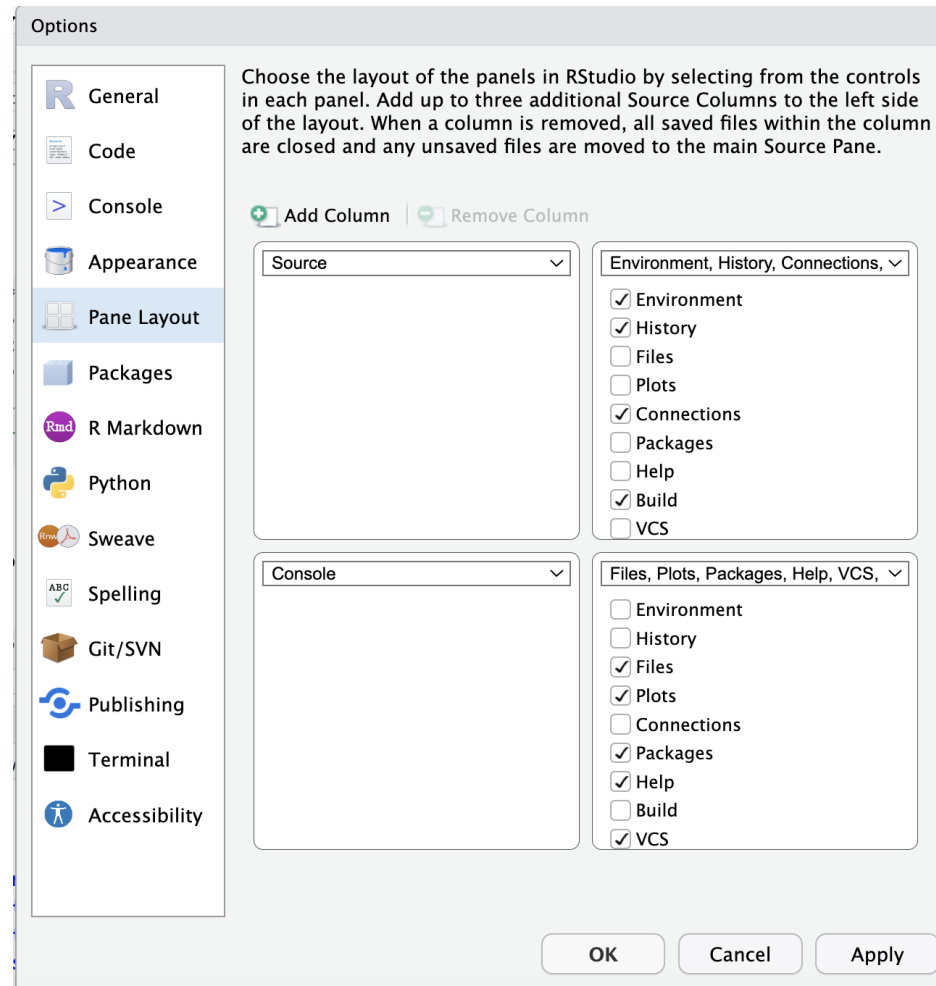
RStudio Layout

If RStudio doesn't look the way you want (or like our RStudio), then:

Click on the pane button, which looks like a waffle with 4 indentations. Scroll down to "Pane Layout".



Default Layout



R Markdown file

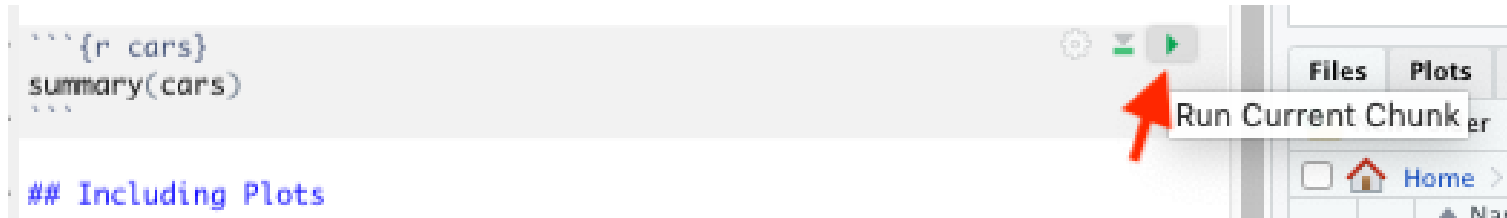
R Markdown files (.Rmd) help generate reports that include your code and output.

1. Helps you describe your code
2. Allows you to check the output
3. Can create many different file types

Code chunks

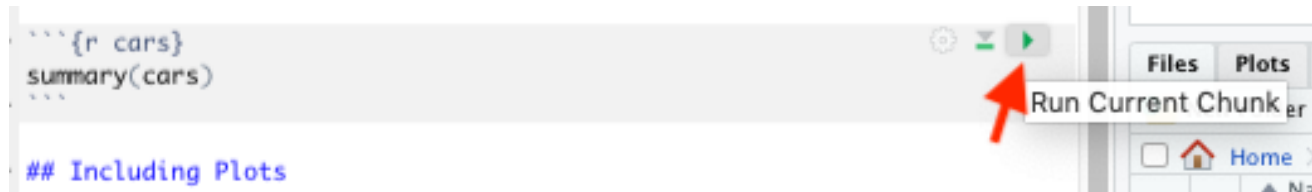
Within R Markdown files are code “chunks”.

This is where you can type R code and run it!



Run code in a chunk

Clicking the run (play) button runs the code in the chunk.



Ctrl + Enter on Windows or Command + Enter on Mac in your script evaluates that line of code

Running a chunk executes the code

- generally see a preview of the output of the code just below the chunk
- see the code in the console

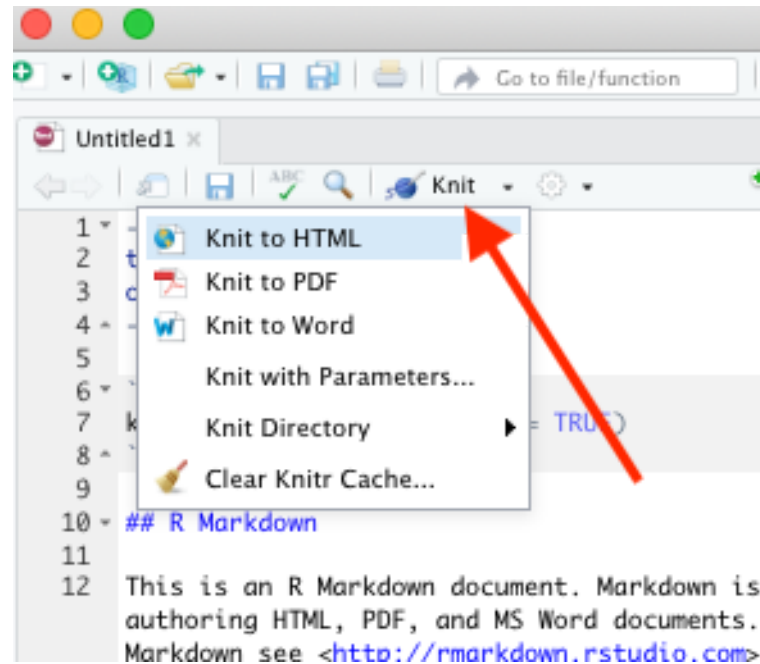
If you get annoyed by code previews in Markdown files...

See the [Help page](#) of the website. You can adjust this and change your RStudio settings:

Tools > Global Options > Appearance

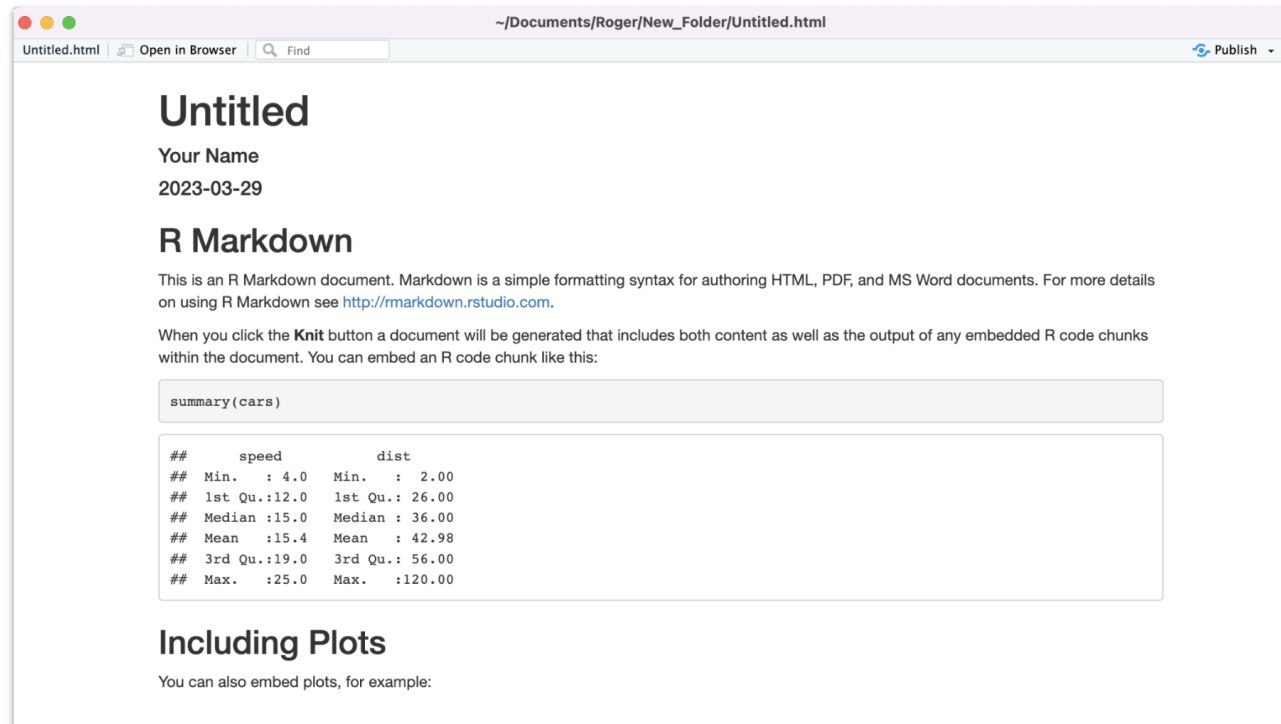
Knit file to html

Running all chunks - this will create a report from the R Markdown document!
Note that it can't use anything not included in the file, it can't use objects in your environment that you were modifying interactively.



Nice report!

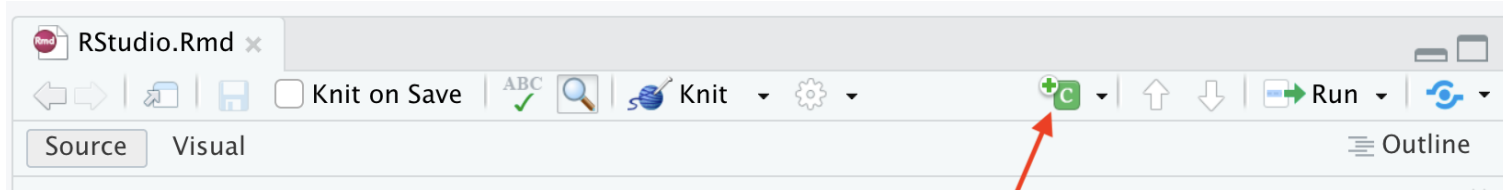
This generates a nice report that you can share with others who can open in any browser.



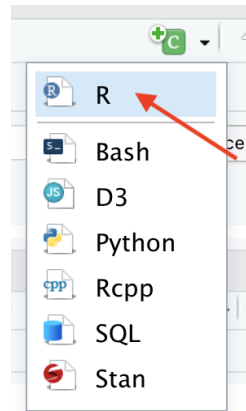
Create Chunks

To create a new R code chunk:

- Use the insert code chunk button at the top of RStudio.



- Select R (default) as the language:



Create Chunks

If you like keyboard shortcuts:

- Windows & Linux use Ctrl+Alt+I
- Mac use Command+Option+I

I is for insert.

Run previous chunks button

You can run all chunks above a specific chunk using this button:

```
```{r, out.width = "80%", echo = FALSE, fig.align='center'}  
knitr::include_graphics("images/chunk.png")
```
```



Errors

R studio can help you find issues in your code. Note that sometimes the error occurs earlier than RStudio thinks.

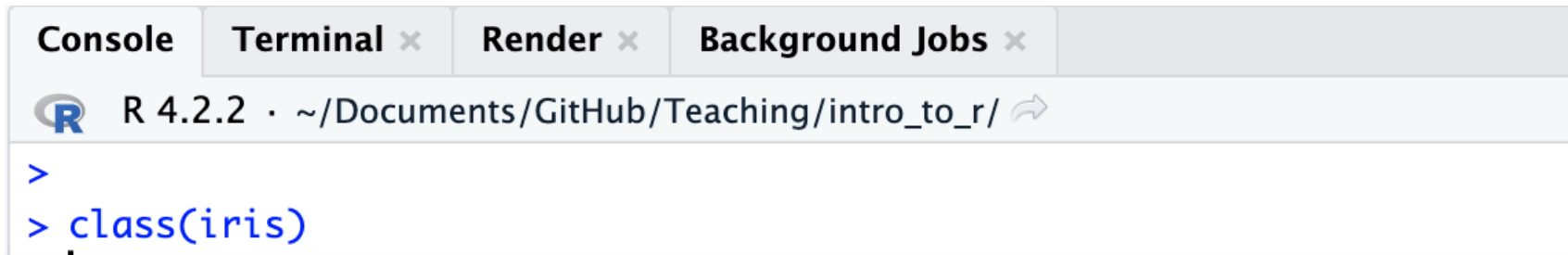


The screenshot shows a snippet of R code in the RStudio editor. Line 305 contains `print(x, ...)` with a yellow highlight. Line 306 contains `{r}`. Line 307 contains `print(x))|` and is marked with a red 'x' icon, indicating an error. A tooltip box is open over line 307, displaying the messages: `unexpected token ')'` and `unexpected end of document`. To the right of the code editor, there are icons for settings (gear), a dropdown menu (triangle), and a run button (green play icon).

```
305 print(x, ...)  
306 {r}  
307 print(x))|  
308 ``````  
3 unexpected token ')'  
3 unexpected end of document
```

Recap of where code goes

- you can test code in the console



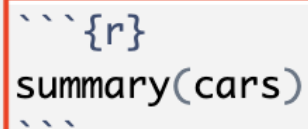
The screenshot shows the RStudio interface with the 'Console' tab selected. The title bar indicates 'R 4.2.2 · ~/Documents/GitHub/Teaching/intro_to_r/'. The console prompt is '>'. The code 'class(iris)' has been entered, and the cursor is positioned at the end of the line.

```
R 4.2.2 · ~/Documents/GitHub/Teaching/intro_to_r/
>
> class(iris)
```

- you can save code in a chunk in the editor (Markdown file)

R Markdown

Code does not go here and instead goes within the grey chunks like this:



The screenshot shows a code chunk within an R Markdown file. The chunk is enclosed in a grey box with a red border. The code inside the chunk is 'summary(cars)'. The chunk is preceded by three backticks and an 'r' in curly braces, and followed by three backticks. On the right side of the chunk, there are three icons: a gear (settings), a downward arrow (collapse), and a rightward arrow (run).

```
```{r}
summary(cars)
```
```

Gut Check

Why are R Markdown files so useful?

1. They let you test your code
2. They let you view the output of your code
3. They let you generate cool reports
4. All of the above

Gut Check

Where does code go typically in an Rmd file?

A

```
```{r}
```

**B**

```
```
```

C

Gut Check

Which button do you click to run the code in a current chunk?

```
```{r}  
library(tidyverse)
```
```



A

B

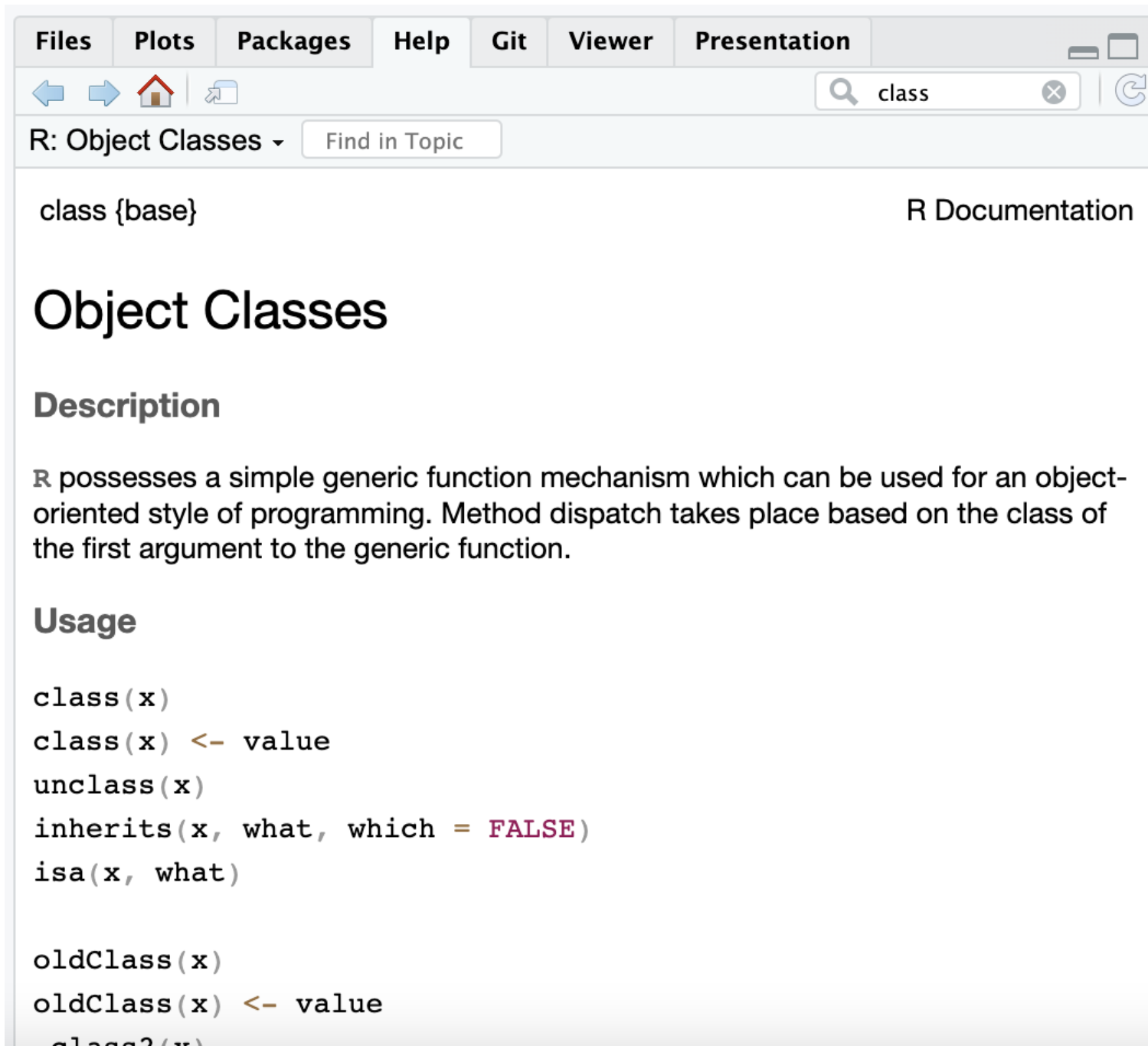
Getting help from the preview

When you type in a function name, a pop up will preview documentation to help you. It also helps you remember the name of the function if you don't remember all of it!

The top screenshot shows the RStudio interface with the search bar containing 'class'. A list of functions is displayed on the left, including 'class', 'class<-', 'classesToAM', 'classInt::', 'classLabel', and 'classMetaName'. The right pane shows the documentation for 'class(x)', titled 'Object Classes', which explains that R possesses a simple generic function mechanism. Below the description, it says 'Press F1 for additional help'.

The bottom screenshot shows the search bar containing 'read'. A list of functions is displayed on the left, including 'read_builtin', 'read_chunk', 'read_csv', 'read_csv2', 'read_csv2_chunked', 'read_csv_chunked', and 'read_delim'. The right pane shows the documentation for 'read_csv()', which lists various arguments like 'file', 'col_names', 'col_types', 'na', 'quoted_na', 'quote', 'comment', 'trim_ws', 'skip', 'n_max', 'guess_max', 'name_repair', 'num_threads', 'readr_threads()', 'progress', 'show_col_types', and 'should_show_types()'. Below the description, it says 'Press F1 for additional help'.

Get help with the help pane



The screenshot shows the RStudio interface with the 'Help' pane active. The top menu bar includes 'Files', 'Plots', 'Packages', 'Help', 'Git', 'Viewer', and 'Presentation'. Below the menu bar, there are navigation icons (back, forward, home, search) and a search bar containing 'class'. The main content area displays the 'R: Object Classes' help page. The page title is 'Object Classes' and the subtitle is 'R Documentation'. The content includes a 'Description' section and a 'Usage' section. The 'Usage' section lists several functions: `class(x)`, `class(x) <- value`, `unclass(x)`, `inherits(x, what, which = FALSE)`, `isa(x, what)`, `oldClass(x)`, `oldClass(x) <- value`, and `class2(x)`.

Files Plots Packages Help Git Viewer Presentation

← → 🏠 🔍 class

R: Object Classes ▾ Find in Topic

class {base} R Documentation

Object Classes

Description

R possesses a simple generic function mechanism which can be used for an object-oriented style of programming. Method dispatch takes place based on the class of the first argument to the generic function.

Usage

```
class(x)
class(x) <- value
unclass(x)
inherits(x, what, which = FALSE)
isa(x, what)

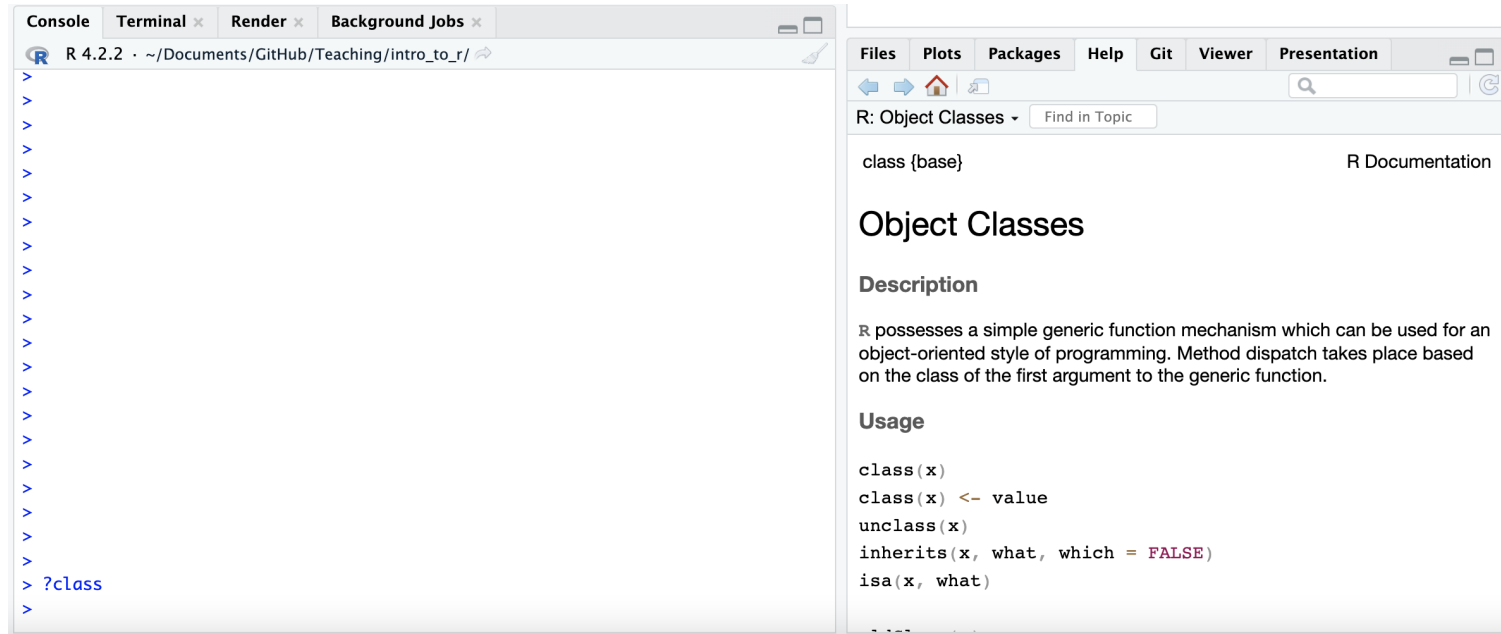
oldClass(x)
oldClass(x) <- value
class2(x)
```

Getting Help with ?

If you know the name of a package or function:

Type `?package_name` or `?function_name` in the console to get information about packages and functions.

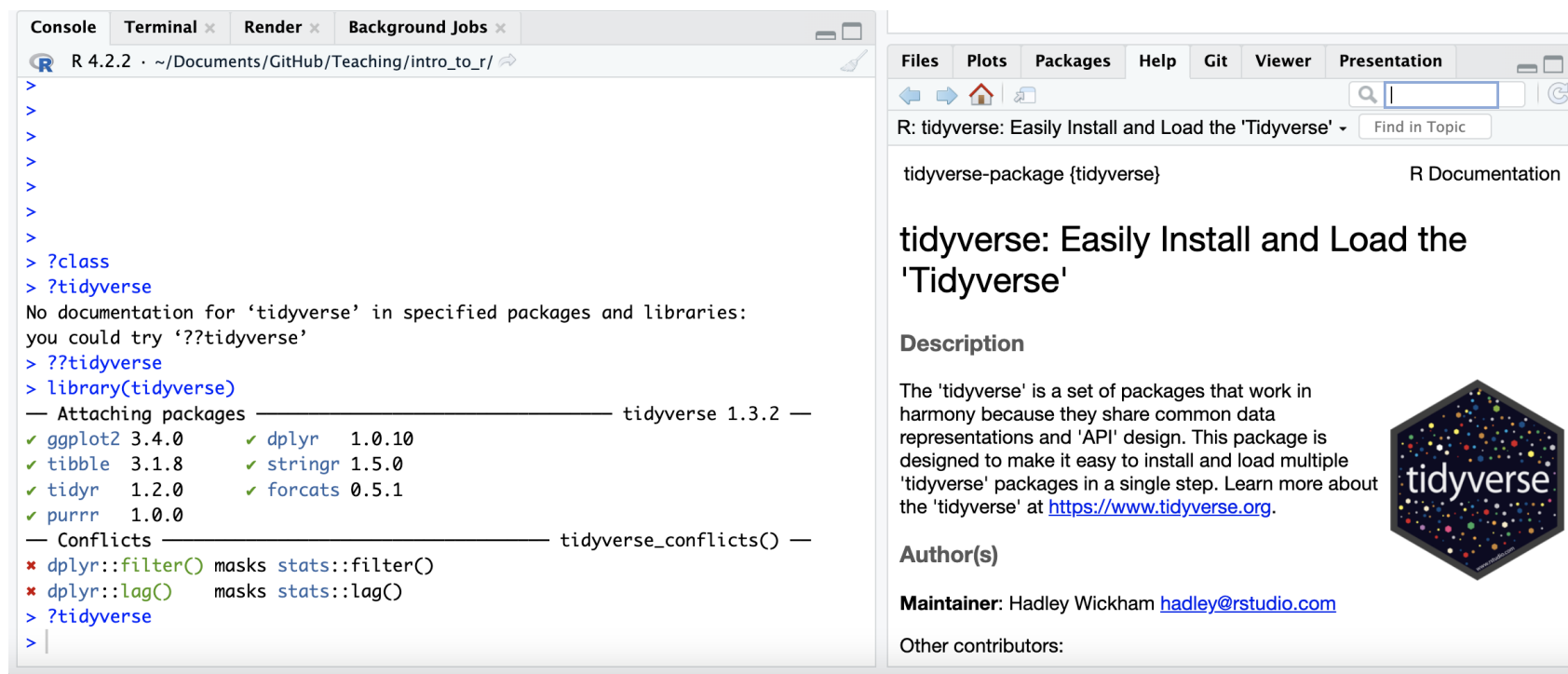
For example: `?readr` or `?read_csv`.



Double Question Mark

If you haven't loaded a package yet into R than you may get a response that there is no documentation.

Typing in `??package_name` can show you packages that you haven't loaded yet.



The screenshot shows the R Studio interface. The console on the left displays the following commands and output:

```
>
>
>
>
>
>
> ?class
> ?tidyverse
No documentation for 'tidyverse' in specified packages and libraries:
you could try '??tidyverse'
> ??tidyverse
> library(tidyverse)
— Attaching packages — tidyverse 1.3.2 —
✓ ggplot2 3.4.0      ✓ dplyr 1.0.10
✓ tibble 3.1.8       ✓ stringr 1.5.0
✓ tidyr 1.2.0        ✓ forcats 0.5.1
✓ purrr 1.0.0
— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag() masks stats::lag()
> ?tidyverse
>
```

The help window on the right shows the documentation for the 'tidyverse' package. The title is 'tidyverse: Easily Install and Load the 'Tidyverse''. The description states: 'The 'tidyverse' is a set of packages that work in harmony because they share common data representations and 'API' design. This package is designed to make it easy to install and load multiple 'tidyverse' packages in a single step. Learn more about the 'tidyverse' at <https://www.tidyverse.org>.' The author is listed as Hadley Wickham, with the email hadley@rstudio.com. The maintainer is also Hadley Wickham. The window also shows the 'tidyverse' logo, which is a hexagon with a starry pattern and the word 'tidyverse' inside.

Summary

- RStudio makes working in R easier
- the Editor (top) is for static code like scripts or R Markdown documents
- The console is for testing code (bottom) - best to save your code though!
- R markdown documents are really helpful for lots of reasons!
- R code goes within what is called a chunk (the gray box with a green play button)

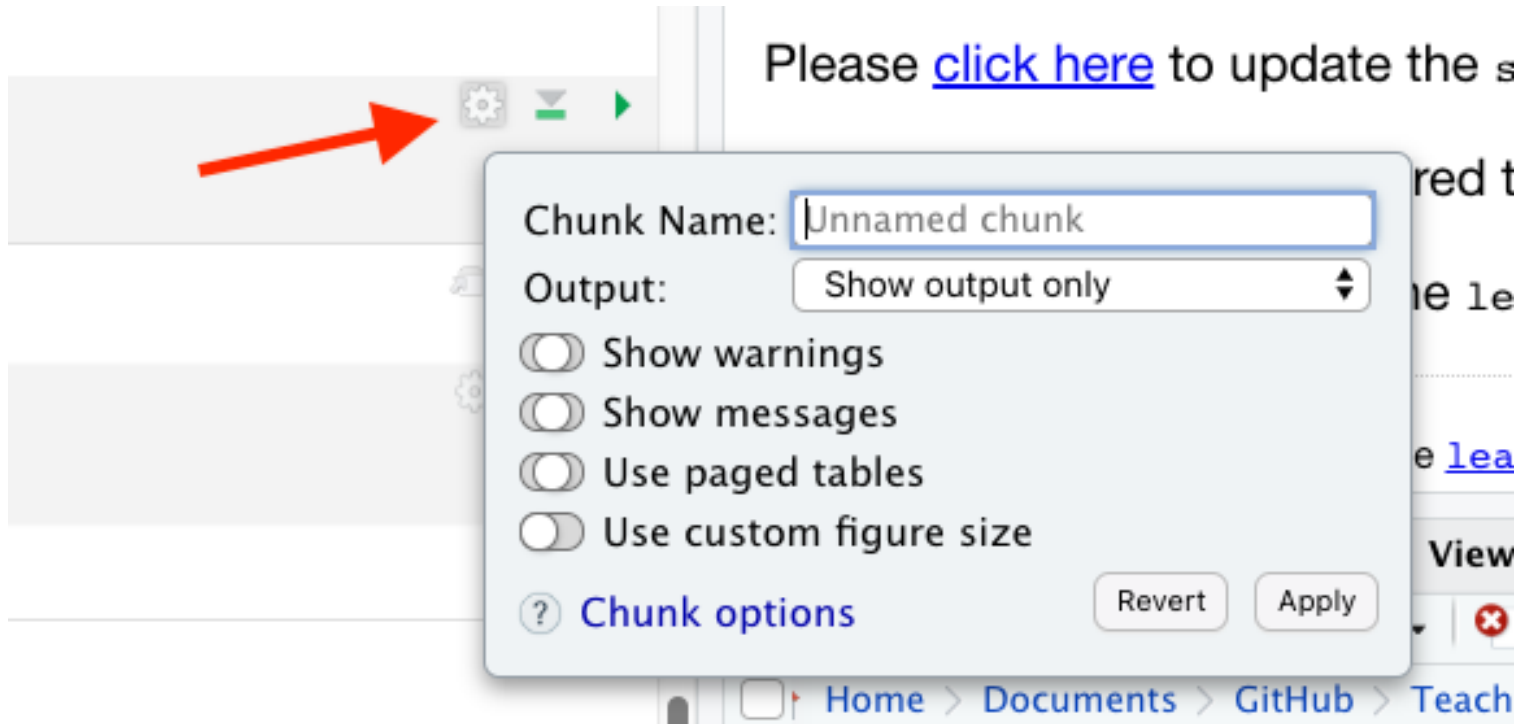
▢ [Class Website](#) ▢ [Lab](#) ▢ [Posit Cheatsheet](#) ▢ [Day 1 Cheatsheet](#)



Image by [Gerd Altmann](#) from [Pixabay](#)

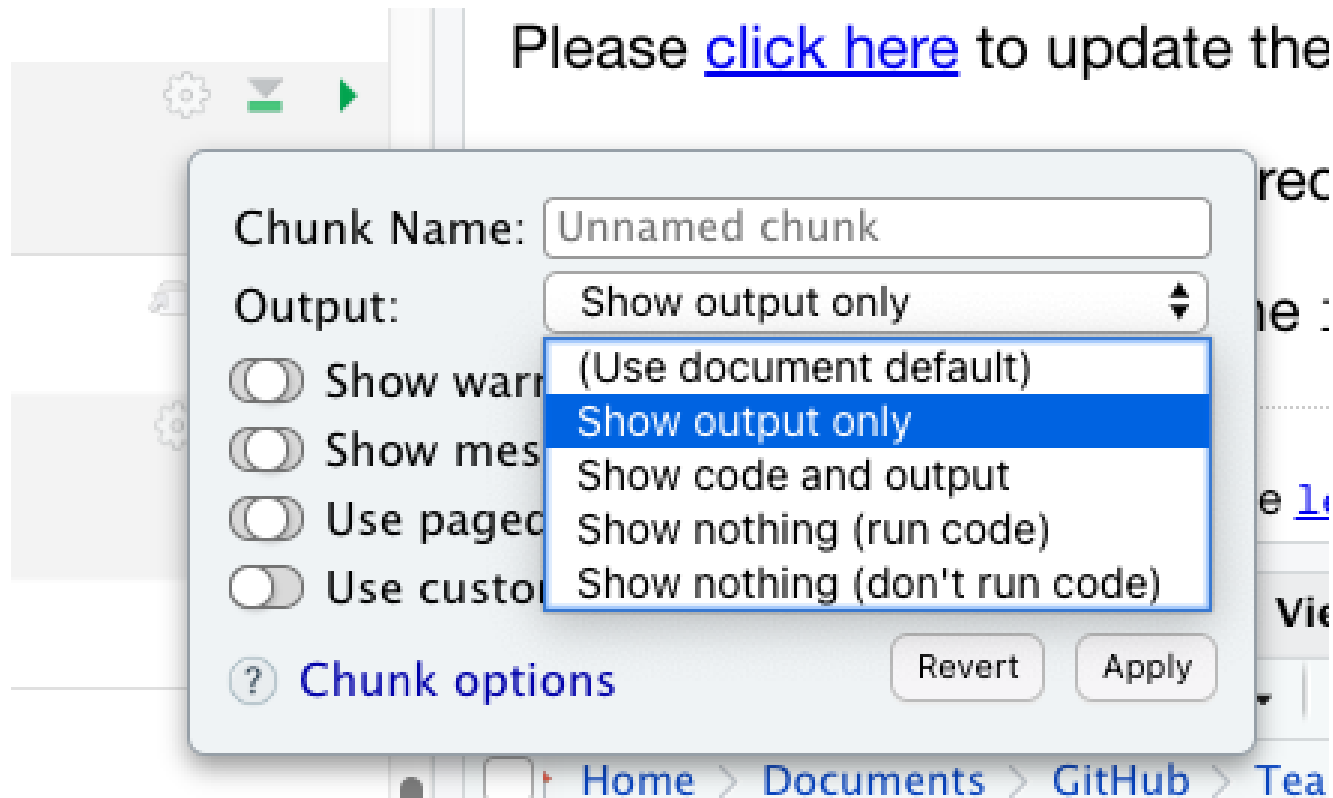
Extra Slides

Chunk settings



Chunk settings

You can specify if a chunk will be seen in the report or not.



Rainbow Parentheses

Tools -> Global Options -> Code -> Display -> Use rainbow parentheses

This can help you see your code more easily.

Press enter to save this setting and get out of this menu.

```
{}{}{
  colourise <- function(text, as = c("success", "skip", "warning", "failure", "error")) {
    if (has_colour()) {
      crayon::style(text, testthat_style(as))
    } else {
      text
    }
  }
}

has_colour <- function() {
  isTRUE(getOption("testthat.use_colours", TRUE)) && crayon::has_color()
}

testthat_style <- function(type = c("success", "skip", "warning", "failure", "error")) {
  type <- match.arg(type)

  c(
    success = "green",
    skip = "blue",
    warning = "magenta",
    failure = "orange",
    error = "orange"
  )[[type]]
}
}{}{}
```